

1. Load the dataset and prepare features for binary classification

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/invistico_Airline1.csv')

# Display the first few rows and basic info
print("Dataset head:")
display(df.head())
print("\nDataset info:")
display(df.info())
```

Dataset head:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure time convenient
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	

5 rows × 22 columns

Dataset info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
```

#	Column	Non-Null Count	Dtype
0	satisfaction	129880 non-null	object
1	Customer Type	129880 non-null	object
2	Age	129880 non-null	int64
3	Type of Travel	129880 non-null	object
4	Class	129880 non-null	object
5	Flight Distance	129880 non-null	int64
6	Seat comfort	129880 non-null	int64
7	Departure/Arrival time convenient	129880 non-null	int64
8	Food and drink	129880 non-null	int64
9	Gate location	129880 non-null	int64
10	Inflight wifi service	129880 non-null	int64
11	Inflight entertainment	129880 non-null	int64
12	Online support	129880 non-null	int64
13	Ease of Online booking	129880 non-null	int64
14	On-board service	129880 non-null	int64
15	Leg room service	129880 non-null	int64
16	Baggage handling	129880 non-null	int64
17	Checkin service	129880 non-null	int64
18	Cleanliness	129880 non-null	int64
19	Online boarding	129880 non-null	int64
20	Departure Delay in Minutes	129880 non-null	int64
21	Arrival Delay in Minutes	129487 non-null	float64

dtypes: float64(1), int64(17), object(4)

memory usage: 21.8+ MB

None

▼ Drop unnecessary columns

Columns like `Unnamed: 0`, `id` are irrelevant for classification.

```
# Drop irrelevant columns
df = df.drop(columns=['Unnamed: 0', 'id'])

print("Dataset head after dropping irrelevant columns:")
display(df.head())
```

Dataset head (no columns dropped as they were not found):

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure time c
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	

5 rows × 22 columns

✓ Handle missing values

Check for missing values and impute or drop them.

```
# Check for missing values
print("Missing values before handling:")
display(df.isnull().sum())

# For 'Arrival Delay in Minutes', we'll impute with the mean for simplicity,
df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].mean(),

print("\nMissing values after handling:")
display(df.isnull().sum())
```


Missing values before handling:

	0
satisfaction	0
Customer Type	0
Age	0
Type of Travel	0
Class	0
Flight Distance	0
Seat comfort	0
Departure/Arrival time convenient	0
Food and drink	0
Gate location	0
Inflight wifi service	0
Inflight entertainment	0
Online support	0
Ease of Online booking	0
On-board service	0
Leg room service	0
Baggage handling	0
Checkin service	0
Cleanliness	0
Online boarding	0
Departure Delay in Minutes	0
Arrival Delay in Minutes	393

✓ Encode categorical features and target variable

Convert categorical features into numerical representations and the target variable 'satisfaction' into binary (0 or 1).

Missing values after handling:

```
from sklearn.preprocessing import LabelEncoder

# Separate target variable
y = df['satisfaction']
X = df.drop('satisfaction', axis=1)

# Encode categorical features ('Customer Type', 'Type of Travel', 'Class')
# and the target variable 'satisfaction'
le = LabelEncoder()
# 'Gender' column was not found in the dataset, removed this line.
X['Customer Type'] = le.fit_transform(X['Customer Type'])
X['Type of Travel'] = le.fit_transform(X['Type of Travel'])
```

```

X['Class'] = le.fit_transform(X['Class'])
y = le.fit_transform(y)

# For ordinal features like 'Ease of Online booking', 'Gate location', etc.,
# they are already numerical and can be treated as such for Decision Trees.

print("Encoded X head:")
display(X.head())
print("\nEncoded y head:")
display(pd.Series(y).head())

```

```

Encoded X head:
  Gate location 0
  Inflight wifi service 0
  Type of Class 0
  Flight Distance 0
  Seat comfort 0
  Departure/Arrival time convenient 0
  Food and drink 0
  Customer Type 0
  Age 65
  Inflight entertainment 1
  Leg room service 1
  Cleanliness 1
  Online boarding 1
  Departure Delay in Minutes 1
  Arrival Delay in Minutes 1
  dtype: int64

```

	Gate location	Inflight wifi service	Type of Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink
0	0	65	1	1	265	0	0
1	0	47	1	0	2464	0	0
2	0	15	1	1	2138	0	0
3	0	60	1	1	623	0	0
4	0	70	1	1	354	0	0

5 rows × 8 columns

```

Encoded y head:
0
dtype: int64

```

2. Split data into training and testing sets for unbiased evaluation

```

from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"X_train shape: {X_train.shape}")

```

```

print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nDistribution of satisfaction in training set:")
print(pd.Series(y_train).value_counts(normalize=True))
print("\nDistribution of satisfaction in test set:")
print(pd.Series(y_test).value_counts(normalize=True))

```

```

X_train shape: (90916, 21)
X_test shape: (38964, 21)
y_train shape: (90916,)
y_test shape: (38964,)

```

Distribution of satisfaction in training set:

```
1    0.547329
```

```
0    0.452671
```

```
Name: proportion, dtype: float64
```

Distribution of satisfaction in test set:

```
1    0.547326
```

```
0    0.452674
```

```
Name: proportion, dtype: float64
```

3. Implement GridSearchCV to optimize `max_depth` and minimum sample splits/leaves

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Define the Decision Tree classifier
dt = DecisionTreeClassifier(random_state=42)

# Define the parameter grid for GridSearchCV
param_grid = {
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8]
}

# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation
    scoring='f1_weighted', # Use f1_weighted for imbalanced classes if any
    n_jobs=-1, # Use all available cores
    verbose=1
)

# Fit GridSearchCV to the training data
print("Starting GridSearchCV...")
grid_search.fit(X_train, y_train)

```

```
# Print the best parameters and best score
print("\nBest parameters found: ", grid_search.best_params_)
print("Best F1-weighted score found: ", grid_search.best_score_)

# Store the best estimator
best_dt_classifier = grid_search.best_estimator_

Starting GridSearchCV...
Fitting 5 folds for each of 80 candidates, totalling 400 fits

Best parameters found: {'max_depth': 20, 'min_samples_leaf': 4, 'min_samples'
Best F1-weighted score found: 0.9393579153655132
```

4. Train the final Decision Tree model using the best hyperparameters and Evaluate Performance

```
from sklearn.metrics import confusion_matrix, f1_score, classification_report

# The best model is already trained and stored in best_dt_classifier from Gr
# Make predictions on the test set
y_pred = best_dt_classifier.predict(X_test)

# Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
display(pd.DataFrame(cm, index=['Actual Not Satisfied', 'Actual Satisfied'],

# Calculate F1-score for the 'Satisfied' class (assuming '1' is the 'Satisfi
f1_satisfied = f1_score(y_test, y_pred, pos_label=1)
print(f"\nF1-score for 'Satisfied' class: {f1_satisfied:.4f}")

# Generate a classification report for overall metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Not Satisfied', 'Satisfied']))
```


Confusion Matrix:

	Predicted Not Satisfied	Predicted Satisfied
Actual Not Satisfied	16723	915
Actual Satisfied	1358	19968

F1-score for 'Satisfied' class: 0.9461

Classification Report:

	precision	recall	f1-score	support
Not Satisfied	0.92	0.95	0.94	17638
Satisfied	0.96	0.94	0.95	21326
accuracy			0.94	38964
macro avg	0.94	0.94	0.94	38964
weighted avg	0.94	0.94	0.94	38964

5. Visualize the Decision Tree and Extract Feature Importance

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Visualize the decision tree
plt.figure(figsize=(20,10))
plot_tree(best_dt_classifier,
          feature_names=X.columns.tolist(),
          class_names=['Not Satisfied', 'Satisfied'],
          filled=True,
          rounded=True,
          fontsize=8)
plt.title("Decision Tree Classifier Visualization")
plt.show()

# Extract and rank feature importance scores
feature_importances = pd.DataFrame({
    'feature': X.columns,
    'importance': best_dt_classifier.feature_importances_
}).sort_values(by='importance', ascending=False)

print("\nTop 10 Feature Importances:")
display(feature_importances.head(10))
```


6. Compare Decision Tree interpretability and performance against Logistic Regression

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Standardize features for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train a Logistic Regression model
log_reg = LogisticRegression(random_state=42, solver='liblinear')
log_reg.fit(X_train_scaled, y_train)

# Make predictions on the scaled test set
y_pred_log_reg = log_reg.predict(X_test_scaled)

# Evaluate Logistic Regression
print("\n--- Logistic Regression Model Evaluation ---")
print("\nConfusion Matrix (Logistic Regression):")
cm_log_reg = confusion_matrix(y_test, y_pred_log_reg)
display(pd.DataFrame(cm_log_reg, index=['Actual Not Satisfied', 'Actual Satisfied'],
                     columns=['Predicted Not Satisfied', 'Predicted Satisfied']))

f1_satisfied_log_reg = f1_score(y_test, y_pred_log_reg, pos_label='Satisfied')
print(f"\nF1-score for 'Satisfied' class (Logistic Regression): {f1_satisfied_log_reg:.4f}")

print("\nClassification Report (Logistic Regression):")
print(classification_report(y_test, y_pred_log_reg, target_names=['Actual Not Satisfied', 'Actual Satisfied']))

# Compare F1-scores
print(f"\nComparison of F1-score for 'Satisfied' class:")
print(f"  Decision Tree: {f1_satisfied:.4f}")
print(f"  Logistic Regression: {f1_satisfied_log_reg:.4f}")

# Interpretability comparison points
print("\n--- Interpretability Comparison ---")
print("Decision Tree: Visualizable (plot_tree), clearly shows feature importance and splits")
print("Logistic Regression: Coefficients indicate direction of influence, but not magnitude or non-linear relationships")

```

```

2                                     Type of Travel    0.025148
--- Logistic Regression Model Evaluation ---
3                                     Class    0.022691
Confusion Matrix (Logistic Regression):
6 Departure/Arrival time convenient    0.020360
11                                     Predicted Not Satisfied Predicted Satisfied
Online support    0.018066
Actual Not Satisfied    14847    2791
Actual Satisfied    3823    17503

F1-score for 'Satisfied' class (Logistic Regression): 0.8411

Classification Report (Logistic Regression):
              precision    recall  f1-score   support

Not Satisfied    0.80    0.84    0.82    17638
Satisfied    0.86    0.82    0.84    21326

   accuracy    0.83    38964
  macro avg    0.83    0.83    0.83    38964
weighted avg    0.83    0.83    0.83    38964

Comparison of F1-score for 'Satisfied' class:
Decision Tree: 0.9461
Logistic Regression: 0.8411

--- Interpretability Comparison ---
Decision Tree: Visualizable (plot_tree), clearly shows decision rules, feature
Logistic Regression: Coefficients indicate direction and magnitude of feature

```

✓ 7. Deliver a clear stakeholder report

Business Comparison: Decision Tree vs. Logistic Regression Trade-offs

Tuning Strategy (Decision Tree): We used `GridSearchCV` to systematically explore combinations of `max_depth`, `min_samples_split`, and `min_samples_leaf`. This comprehensive search ensured that the model was optimized to prevent overfitting (by limiting complexity) and improve generalization on unseen data, focusing on balancing precision and recall for both satisfaction classes using `f1_weighted`.

Evaluation Rationale: The F1-score for the 'Satisfied' class was chosen as a primary metric to balance precision (correctly identifying satisfied customers) and recall (capturing all satisfied customers). This is crucial for airline management as misclassifying a dissatisfied customer as satisfied, or vice-versa, can have significant business implications.